Unit Testing

Website TV Desa Kota Batu

Version	By	Description
01	Muhammad Rafid Mursyaday Putra Satria	This Unit Testing using PHPUnit

Judul	Login
Pengujian	

```php
<?php

namespace Tests\Feature;

use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Foundation\Testing\WithFaker;
use Tests\TestCase;
use App\Models\User;
use Illuminate\Support\Facades\Hash;

class LoginTest extends TestCase
{
    use RefreshDatabase;

    protected $user;
    protected $loginData;

    protected function setUp(): void
    {
        parent::setUp();

        $this->user = User::factory()->create([
            'email' => 'test@example.com',
            'password' => Hash::make('password123'),
            'role' => 'visitor'
        ]);

        $this->loginData = [
            'email' => 'test@example.com',
            'password' => 'password123'
        ];
    }

    public function test_visitor_login_valid()
    {
        $response = $this->post(route('visitor.login'), $this->loginData);

        $response->assertRedirect(route('home'));
        $this->assertAuthenticated();
        $this->assertAuthenticatedAs($this->user);
    }

    public function test_visitor_login_invalid()
    {
        $invalidData = [
            'email' => 'test@example.com',
            'password' => 'wrongpassword'
        ];

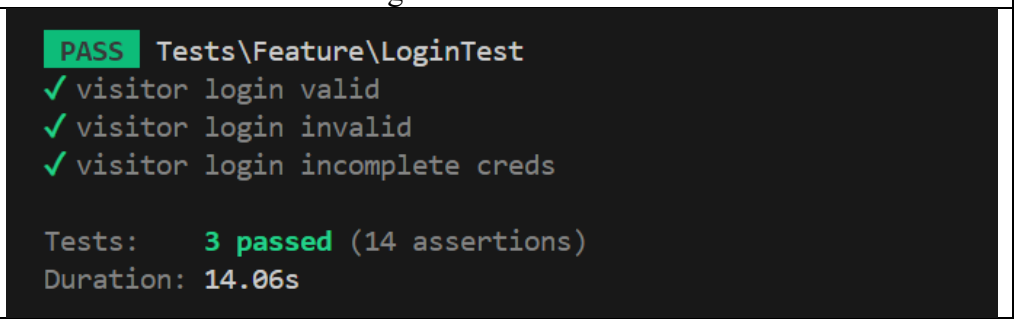
        $response = $this->post(route('visitor.login'), $invalidData);

        $response->assertStatus(302);
        $response->assertSessionHasErrors([
            'error' => 'Invalid email or password. Please check your credentials.'
        ]);
        $this->assertGuest();
    }

    public function test_visitor_login_incomplete_creds()
    {
        $incompleteData = [
            'email' => 'test@example.com',
            'password' => ''
        ];

        $response = $this->post(route('visitor.login'), $incompleteData);

        $response->assertStatus(302);
        $response->assertSessionHasErrors([
            'error' => 'Password is required'
        ]);
        $this->assertGuest();
    }
```

Keterangan	Pada unit testing Login ini, dilakukan pengujian : - Login Valid - Login Invalid - Salah satu form kosong
Hasil	

Judul	Register
Pengujian	

```php
<?php

namespace Tests\Feature;

use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Foundation\Testing\WithFaker;
use Tests\TestCase;
use App\Models\User;

class RegisterTest extends TestCase
{
    use RefreshDatabase, WithFaker;

    public function test_visitor_can_register_with_valid_data()
    {
        $userData = [
            'name' => 'Test User',
            'email' => 'test@example.com',
            'password' => 'password123'
        ];

        $response = $this->post(route('visitor.register'), $userData);

        $response->assertRedirect(route('home'));
        $this->assertDatabaseHas('users', [
            'email' => 'test@example.com',
            'name' => 'Test User',
            'role' => 'visitor'
        ]);
        $this->assertAuthenticated();
    }

    public function test_visitor_cannot_register_with_existing_email()
    {
        User::factory()->create([
            'email' => 'test@example.com'
        ]);

        $userData = [
            'name' => 'Test User',
            'email' => 'test@example.com',
            'password' => 'password123'
        ];

        $response = $this->post(route('visitor.register'), $userData);

        $response->assertStatus(302);
        $response->assertSessionHasErrors([
            'error' => 'This email is already registered'
        ]);
    }

    public function test_visitor_cannot_register_with_invalid_email_format()
    {
        $userData = [
            'name' => 'Test User',
            'email' => 'invalid-email',
            'password' => 'password123'
        ];

        $response = $this->post(route('visitor.register'), $userData);

        $response->assertStatus(302);
        $response->assertSessionHasErrors([
            'error' => 'Please enter a valid email address'
        ]);
    }

    public function test_visitor_cannot_register_with_short_password()
    {
        $userData = [
            'name' => 'Test User',
            'email' => 'test@example.com',
            'password' => 'short'
        ];

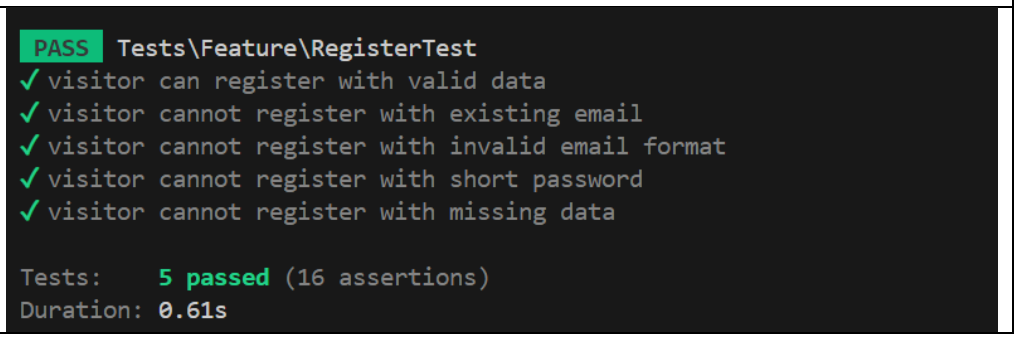
        $response = $this->post(route('visitor.register'), $userData);

        $response->assertStatus(302);
        $response->assertSessionHasErrors([
            'error' => 'Password must be at least 8 characters'
        ]);
    }

    public function test_visitor_cannot_register_with_missing_data()
    {
        $userData = [
            'name' => '',
            'email' => 'test@example.com',
            'password' => 'password123'
        ];

        $response = $this->post(route('visitor.register'), $userData);

        $response->assertStatus(302);
        $response->assertSessionHasErrors([
            'error' => 'Name is required'
        ]);
    }
}
```

Keterangan	Pada unit testing registrasi ini, dilakukan pengujian : - Register dengan data valid - Register dengan email yang sudah terdaftar - Register dengan format email salah - Register dengan pasword kurang dari 8 karakter - Register dengan form kosong
Hasil	

Judul	Tambah Berita
Pengujian	
Keterangan	Pada unit testing registrasi ini, dilakukan pengujian : - Berhasil menambah berita dengan data yang valid - Menambah berita dengan deskripsi kurang dari 50 - Menambah berita dengan image invalid - Menambah berita dengan salahsatu form kosong

```php
<?php

namespace Tests\Feature;

use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Foundation\Testing\WithFaker;
use Tests\TestCase;
use Illuminate\Support\Facades\Log;
use Illuminate\Http\UploadedFile;
use Illuminate\Support\Facades\Storage;

class TambahTest extends TestCase
{
    use RefreshDatabase, WithFaker;

    protected function setUp(): void
    {
        parent::setUp();
        Storage::fake('public');
    }
    public function test_can_store_citizen_news_with_valid_data()
    {
        $validData = [
            'title' => 'Test News Title',
            'description' => 'This is a test description that meets the minimum length requirement of 50 characters for validation.',
            'teaser' => 'This is a test teaser',
            'image' => UploadedFile::fake()->image('news.jpg'),
            'author' => 'Test Author',
            'category' => 'KotaTerkini'
        ];

        $response = $this->post(route('CreateBerita'), $validData);

        $response->assertRedirect('/citizen');
    }
    public function test_cannot_store_news_with_short_description()
    {
        $invalidData = [
            'title' => 'Test News Title',
            'description' => 'Too short',
            'teaser' => 'This is a test teaser',
            'image' => UploadedFile::fake()->image('news.jpg'),
            'author' => 'Test Author',
            'category' => 'KotaTerkini'
        ];

        $response = $this->post(route('CreateBerita'), $invalidData);

        $response->assertStatus(302);
        $response->assertSessionHasErrors([
            'error' => 'Failed to submit news: News content must be at least 50 characters'
        ]);
    }
    public function test_cannot_store_news_with_invalid_image_type()
    {
        $invalidData = [
            'title' => 'Test News Title',
            'description' => 'This is a test description that meets the minimum length requirement of 50 characters for validation.',
            'teaser' => 'This is a test teaser',
            'image' => UploadedFile::fake()->create('document.pdf', 1000),
            'author' => 'Test Author',
            'category' => 'KotaTerkini'
        ];

        $response = $this->post(route('CreateBerita'), $invalidData);

        $response->assertStatus(302);
        $response->assertSessionHasErrors('error');
    }
    public function test_cannot_store_news_with_missing_required_fields()
    {
        $invalidData = [
            'title' => '',
            'description' => '',
            'teaser' => '',
            'author' => '',
            'category' => ''
        ];

        $response = $this->post(route('CreateBerita'), $invalidData);

        $response->assertStatus(302);
        $response->assertSessionHasErrors(['error']);
    }
}

```

Hasil	 PASS Tests\Feature\TambahTest ✓ can store citizen news with valid data ✓ cannot store news with short description ✓ cannot store news with invalid image type ✓ cannot store news with missing required fields Tests: 4 passed (11 assertions) Duration: 1.32s

Judul	Edit Berita
Pengujian	

```php
<?php

namespace Tests\Feature;

use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Foundation\Testing\WithFaker;
use Tests\TestCase;
use Illuminate\Http\UploadedFile;
use Illuminate\Support\Facades\Storage;
use App\Models\Berita;

class UpdateTest extends TestCase
{
    use RefreshDatabase, WithFaker;

    protected function setUp(): void
    {
        parent::setUp();
        Storage::fake('public');
    }

    public function test_can_update_citizen_news_with_valid_data()
    {
        $validData = [
            'title' => 'Test News Title',
            'description' => 'This is a test description that meets the minimum length requirement of 50 characters for validation.',
            'teaser' => 'This is a test teaser',
            'image' => UploadedFile::fake()->image('news.jpg'),
            'author' => 'Test Author',
            'category' => 'KotaTerkini'
        ];

        $response = $this->post(route('CreateBerita'), $validData);

        $createdBerita = \App\Models\Berita::latest()->first();

        $updateData = [
            'title' => 'Updated Title',
            'description' => 'This is an updated description that meets the minimum length requirement of 50 characters for validation.',
            'teaser' => 'This is an updated teaser',
            'image' => UploadedFile::fake()->image('updated.jpg'),
            'author' => 'Updated Author',
            'category' => 'KotaTerkini'
        ];

        $response = $this->put(route('UpdateBerita', $createdBerita->id), $updateData);

        $response->assertRedirect('/admin/kotaterkini');

        $this->assertDatabaseHas('beritas', [
            'id' => $createdBerita->id,
            'title' => 'Updated Title',
            'author' => 'Updated Author'
        ]);
    }

    public function test_cannot_update_news_with_short_description()
    {
        $validData = [
            'title' => 'Test News Title',
            'description' => 'This is a test description that meets the minimum length requirement of 50 characters for validation.',
            'teaser' => 'This is a test teaser',
            'image' => UploadedFile::fake()->image('news.jpg'),
            'author' => 'Test Author',
            'category' => 'KotaTerkini'
        ];

        $response = $this->post(route('CreateBerita'), $validData);

        $createdBerita = \App\Models\Berita::latest()->first();
        $updateData = [
            'title' => 'Updated Title',
            'description' => 'Short description.',
            'teaser' => 'This is an updated teaser',
            'image' => UploadedFile::fake()->image('updated.jpg'),
            'author' => 'Updated Author',
            'category' => 'KotaTerkini'
        ];

        $response = $this->put(route('UpdateBerita', $createdBerita->id), $updateData);

        $response->assertStatus(302);
    }

    public function test_cannot_update_news_with_invalid_image_type()
    {
        $validData = [
            'title' => 'Test News Title',
            'description' => 'This is a test description that meets the minimum length requirement of 50 characters for validation.',
            'teaser' => 'This is a test teaser',
            'image' => UploadedFile::fake()->image('news.jpg'),
            'author' => 'Test Author',
            'category' => 'KotaTerkini'
        ];
        $response = $this->post(route('CreateBerita'), $validData);

        $createdBerita = \App\Models\Berita::latest()->first();

        $updateData = [
            'title' => 'Updated Title',
            'description' => 'This is an updated description that meets the minimum length requirement of 50 characters.',
            'teaser' => 'This is an updated teaser',
            'image' => UploadedFile::fake()->create('invalid.txt', 10),
            'author' => 'Updated Author',
            'category' => 'KotaTerkini'
        ];

        $response = $this->put(route('UpdateBerita', $createdBerita->id), $updateData);

        $response->assertStatus(302);
    }

    public function test_cannot_update_news_with_missing_required_fields()
    {
        $validData = [
            'title' => 'Test News Title',
            'description' => 'This is a test description that meets the minimum length requirement of 50 characters for validation.',
            'teaser' => 'This is a test teaser',
            'image' => UploadedFile::fake()->image('news.jpg'),
            'author' => 'Test Author',
            'category' => 'KotaTerkini'
        ];

        $response = $this->post(route('CreateBerita'), $validData);

        $createdBerita = \App\Models\Berita::latest()->first();

        $updateData = [
            'title' => 'Updated Title',
            'description' => 'This is an updated description that meets the minimum length requirement of 50 characters.',
            'teaser' => 'This is an updated teaser',
            'image' => UploadedFile::fake()->image('updated.jpg'),
            'author' => '',
            'category' => '',
        ];

        $response = $this->put(route('UpdateBerita', $createdBerita->id), $updateData);

        $response->assertStatus(302);
        $response->assertSessionHasErrors(['author', 'category']);
    }
}
```

Keterangan	Pada unit testing registrasi ini, dilakukan pengujian : - Update data dengan data baru valid - Update data dengan deskripsi baru kurang dari 50 - Update data dengan image baru tidak valid - Update data dengan salah satu form dikosongkan
Hasil	